# Pragna — Detailed Daily Timeline

*AI & DS Knowledge Assistant · Backend · RAG + Gemini · 4 weeks (20 working days)*

You own the engine: reading the PDFs, retrieving the right text, generating answers with sources, and producing summaries, questions and quizzes with Gemini.

## What you are responsible for

- Python environment, API keys and configuration
- PDF extraction, chunking and SBERT embeddings
- Vector index and top-k retrieval with page references
- RAG answer chain with citations and fallback
- Gemini summaries, questions and quizzes
- Caching, error handling and deployment

## Tools and technology

- Python, PyMuPDF
- sentence-transformers (SBERT), FAISS or ChromaDB
- Gemini API (google-generativeai)
- Streamlit Cloud, Git

## Hand-offs with teammates

- From Nishant: cleaned documents and curriculum data (D2–D7), test set (D8)
- To Preethi: answer, summary, question and quiz functions (D6–D13)
- To Nishant: retrieval results for evaluation

**Milestones: D5 · D10 · D15 · D20 (details on the weekly pages)**

# Pragna · Week 1 · Foundations

| Day | What to do (in detail) | Tools | Output / done when |
|---|---|---|---|
| **D1** | Set up the Python environment and requirements.txt (streamlit, sentence-transformers, faiss, google-generativeai, networkx, PyMuPDF). Get the Gemini key and store it in .env / Streamlit secrets. Choose the embedding model and vector store. | Python, venv, .env | Environment runs; key stored safely; stack decided |
| **D2** | Write PDF text extraction that keeps page numbers. Prototype chunking (about 500 tokens with overlap) and test it on one syllabus PDF. | PyMuPDF | Chunks with page numbers from a sample PDF |
| **D3** | Generate SBERT embeddings (all-MiniLM-L6-v2) for the chunks and build a vector index. Save the index to disk and reload it. | sentence-transformers, FAISS | Saved, reloadable vector index |
| **D4** | Write retrieve(query, k) returning top-k chunks with file, page and similarity score. Test with 10 sample queries and judge relevance by hand. | FAISS, Python | Retrieval returns relevant chunks with pages |
| **D5** | Build a Gemini wrapper with retries, timeout and error handling. Write prompt templates for answers, summaries, questions and quizzes. | Gemini API | Working wrapper and 4 prompt templates |

**D5 — Retrieval returns correct chunks; Gemini wrapper ready**

# Pragna · Week 2 · Core Loop

| Day | What to do (in detail) | Tools | Output / done when |
|---|---|---|---|
| **D6** | Build the RAG chain: retrieve chunks, build the prompt with context, call Gemini and return the answer plus a list of sources the UI can show. | Python, Gemini API | answer(question, mode) function |
| **D7** | Add citation rules to the prompt (answer only from context, cite the page). Add a "not found in documents" fallback when similarity is below a threshold. | Prompt design | Source-backed answers; no guessing |
| **D8** | Add syllabus Q&A mode with metadata filters for the chosen subject and module. Make regulation mode search only regulation documents. | FAISS filters / Chroma metadata | Module-scoped and regulation modes |
| **D9** | Review failed cases from Nishant's test set. Tune chunk size, top-k, threshold and prompt wording, then re-run and compare. | Test set, spreadsheet | Better accuracy with changes logged |
| **D10** | Fix remaining retrieval and prompt problems from integration. Freeze the answer function signature so the UI does not break and write short API notes. | Python | Stable chatbot backend; API notes |

**D10 — RAG chatbot answers with page references (≥80% on test set)**

# Pragna · Week 3 · Module Intelligence + Network

| Day | What to do (in detail) | Tools | Output / done when |
|---|---|---|---|
| **D11** | Write generate_summary(module): retrieve module content and prompt Gemini for a structured summary (key points, definitions, formulas). Test on 5 modules. | Gemini API, prompts | Summary function tested on 5 modules |
| **D12** | Write generate_questions(module, type, n) for short and long answers with model answers. Control difficulty through the prompt. | Gemini API | Question generator |
| **D13** | Write generate_quiz(module, n): MCQs with 4 options, correct answer and explanation as JSON. Validate the JSON and retry on invalid output. | json, retry logic | Reliable quiz generator |
| **D14** | Cache generated outputs by module and type to save API calls. Handle rate limits and API failures with retry and friendly error codes. | Local cache, try/except | Faster, cheaper, safer calls |
| **D15** | Connect every feature to the UI functions, run end-to-end tests across several modules and fix integration bugs. | Test checklist | All AI features work end to end |

**D15 — All AI features working end to end**

# Pragna · Week 4 · Harden & Launch

| Day | What to do (in detail) | Tools | Output / done when |
|---|---|---|---|
| **D16** | Optimise latency: load the index once, batch embeddings and cache results. Add a fallback message or cached answer when Gemini fails. | st.cache_resource | Faster responses; graceful fallback |
| **D17** | Fix backend bugs from the student usability test. Check answer quality on 10 new unseen questions. | Test questions | Bugs fixed; quality confirmed |
| **D18** | Deploy on Streamlit Community Cloud: push the repo, add secrets, confirm index files load and test the live link. | Streamlit Cloud, GitHub | Live public URL |
| **D19** | Run a final regression of every feature on the deployed app across different modules. Freeze the code and tag a release. | Git tags | Stable tagged release |
| **D20** | Rehearse the demo, check the deployed app before presenting, and submit code and documentation. | Deployed app, README | Code + docs submitted |

**D20 — Deployed app and final demo**